

DANKE MOTORSPORT

Review de Requisitos — Estado Atualizado

Baseado no estado do repositório em 06/06/2026 | branch main

#	Critério	Peso	Status
1	Acesso apenas após autenticação	0,5 pt	■ CONCLUÍDO
2	Cadastro de novos usuários	0,5 pt	■ CONCLUÍDO
3	Atualização de cadastro (autenticada)	0,5 pt	■■ PARCIAL
4	Base de dados no back-end	0,5 pt	■ CONCLUÍDO
5	Compartilhar dados entre usuários	0,75 pt	■ CONCLUÍDO
6	Front-end responsivo (desktop e mobile)	1,0 pt	■■ PARCIAL
7	Deploy 24/7 em servidor	0,75 pt	■■ PARCIAL
8	Organização da entrega e código	1,0 pt	■■ PARCIAL
9	Documentação do código	0,5 pt	■ AUSENTE
	PONTOS CONCLUÍDOS: 2,25 pt EM RISCO: 3,75 pt	6,0 pt	Evolução: +1,75 pt vs. review anterior

Critério 1 — Autenticação ■ (0,5 pt)

■ CONCLUÍDO. AuthController com JWT, BCrypt.Verify, ProtectedRoute com roles, redirecionamento automático por perfil, tela de loading durante verificação.

O que está implementado

- POST /danke/auth/login valida e-mail + BCrypt em Clientes e Funcionarios.
- Token JWT com claims de Id, Role, Nome, Email. Expiração 7 dias.
- Frontend: ProtectedRoute bloqueia acesso a /client-dashboard e /employee-dashboard sem token.
- AuthContext persiste sessão via localStorage e reidrata ao recarregar a página.

Pendências para nota máxima

■■ Chave JWT com fallback hardcoded no AuthController e no Program.cs. Solução: lançar InvalidOperationException se Jwt:Key for nulo, forçando configuração correta.

Critério 2 — Cadastro de novos usuários ■ (0,5 pt)

■ CONCLUÍDO. POST /danke/clientes com BCrypt.HashPassword + formulário de cadastro funcional no Auth.jsx com validação de senhas iguais.

Pendências para nota máxima

■■ Registro disponível apenas para Clientes. Funcionarios não podem se auto-cadastrar (exige inserção direta no banco). Aceitável para o escopo do projeto se documentado.

Critério 3 — Atualização de cadastro ■■ (0,5 pt)

■■ PARCIAL. PUT /danke/clientes/{id} existe no backend mas tem dois problemas críticos.

O que falta

- **Problema 1 — Sem [Authorize]:** o endpoint PUT não está protegido. Qualquer pessoa, sem estar logada, pode enviar uma requisição e alterar qualquer cliente.
- **Problema 2 — Sem verificação de ownership:** um cliente logado como id=1 pode fazer PUT /danke/clientes/99 e alterar os dados de outro usuário.
- **Problema 3 — Sem página no frontend:** não existe uma página de perfil/edição acessível pelo ClientDashboard. O avaliador não consegue testar essa funcionalidade.

Correções necessárias (backend)

```
[HttpPut("{id}")]
[Authorize] // <-- adicionar
public IActionResult AtualizarCliente(int id, ...) {
    var myId = int.Parse(User.FindFirst(ClaimTypes.NameIdentifier)!.Value);
    if (myId != id) return Forbid(); // <-- adicionar
```

```
...  
}
```

Critério 4 — Base de dados no back-end ■ (0,5 pt)

■ CONCLUÍDO (inalterado desde o primeiro review). EF Core + PostgreSQL + Supabase com 3 tabelas relacionais e migrations aplicadas.

Critério 5 — Compartilhar dados entre usuários ■ (0,75 pt)

■ CONCLUÍDO. Clientes criam revisões (POST). Funcionários visualizam revisões de clientes (GET /pendentes com Include do Cliente) e atualizam status (PATCH). Dados transitam entre perfis de forma controlada por role.

Detalhe do fluxo

- Cliente faz agendamento → revisão criada com status 'Pendente', IdFuncionario null.
- Funcionário vê card no Kanban (coluna Pendentes) com nome e dados do cliente.
- Funcionário clica Iniciar → PATCH define status 'Em Andamento' e atribui IdFuncionario.
- Funcionário clica Concluir → PATCH define status 'Concluído' e DatFinalizacao.
- Cliente vê status atualizado na tabela de histórico do seu dashboard.

Critério 6 — Front-end responsivo ■■ (1,0 pt)

■■ PARCIAL. LandingPage e Navbar têm @media queries para 768px. Auth, ClientDashboard e EmployeeDashboard não têm.

O que falta

- **Auth.css:** .inpt-form ainda usa width: 25vw — ilegível em celular.
- **ClientDashboard.css:** nenhum @media query. Tabela de histórico quebra em telas pequenas.
- **EmployeeDashboard.css:** Kanban em 3 colunas fixas — não colapsa para mobile.

Correção mínima necessária

```
/* Auth.css */  
@media (max-width: 768px) {  
  .inpt-form { width: 85vw; }  
}  
  
/* ClientDashboard.css */  
@media (max-width: 768px) {  
  .history-table { font-size: 12px; }  
  .table-responsive { overflow-x: auto; }  
}  
  
/* EmployeeDashboard.css — colunas do Kanban */  
@media (max-width: 768px) {
```

```
.kanban-board { flex-direction: column; }  
}
```

Critério 7 — Deploy 24/7 ■■ (0,75 pt)

■■ PARCIAL. Keep-alive do banco configurado (excelente), mas o frontend e o backend não estão hospedados ainda.

- GitHub Actions keep-alive a cada 5 dias — Supabase não será pausado.
- Frontend: não há deploy no Vercel ainda.
- Backend: nenhuma plataforma de hospedagem configurada (Railway, Render, Azure, etc.).
- CORS em Program.cs só permite localhost:5173. Precisa incluir a URL do Vercel antes do deploy.

Próximo passo para fechar esse critério

1. Deploy do frontend no Vercel (conectar repositório GitHub → definir VITE_API_URL).
2. Deploy do backend no Railway ou Render (definir SUPABASE_CONNECTION_STRING e JWT_KEY como env vars).
3. Atualizar CORS em Program.cs com a URL real do Vercel.

Critério 8 — Organização do código e entrega ■■ (1,0 pt)

■■ PARCIAL. A estrutura de pastas e a mensagens de commit (feat/fix/security) são boas, mas há 4 problemas que o avaliador vai notar.

Problemas encontrados

■ frontend/dist/ commitado no repositório. Adicionar ao .gitignore e remover com `git rm --cached -r frontend/dist/`

■ Endpoints de Clientes e Funcionarios sem [Authorize] — falha de segurança arquitetural que demonstra falta de atenção ao design do sistema.

■■ Imports não usados em main.jsx (Route, Router). Pequeno mas visível.

■■ Inconsistência de naming: Cliente usa Id (int), Funcionario usa IdFuncionario (int). Modelos do mesmo sistema devem ter convenção uniforme.

Critério 9 — Documentação do código ■ (0,5 pt)

■ AUSENTE. Nenhum XML doc comment no C# e nenhum JSDoc no React. Os comentários existentes são didáticos, não de documentação profissional.

O que adicionar — C# (mínimo aceitável)

```
/// <summary>Autentica um usuário e retorna token JWT.</summary>  
/// <param name='request'>E-mail e senha do usuário.</param>
```

```
/// <returns>Token JWT + dados básicos do usuário.</returns>
[HttpPost("login")]
public IActionResult Login([FromBody] LoginRequest request) { ... }
```

O que adicionar — React (mínimo aceitável)

```
/**
 * Dashboard do cliente. Exibe formulário de agendamento
 * e histórico de revisões solicitadas.
 * Requer autenticação com role 'Cliente'.
 */
function ClientDashboard() { ... }
```

Adicione doc comments nos 4 controllers (C#) e nos 4 componentes de página (React). Estimativa: 30 minutos de trabalho para fechar 0,5 ponto.

Resumo Executivo — Prioridades Restantes

#	Ação	Pontos	Tempo est.
1	Adicionar [Authorize] + verificação de ownership em ClientesController e RevisõesController	0,5 pt (req. 2)	20 min
2	Criar página de Perfil no ClientDashboard (formulário de edição)	0,5 pt (req. 3)	45 min
3	Media queries em Auth.css, ClientDashboard.css, EmployeeDashboard.css	1,5 pt (req. 6)	40 min
4	Deploy: Vercel (frontend) + Railway (backend) + atualizar CORS	0,75 pt (req. 7)	1–2 h
5	XML doc comments nos controllers C# + JSDoc nos componentes React	0,5 pt (req. 9)	30 min
6	Remover frontend/dist/ do git + limpar imports não usados em main.jsx	Req. 8	10 min
7	Remover chave JWT hardcoded — lançar erro se Jwt:Key não configurada	Segurança	10 min

Com as 7 ações acima concluídas (~3,5 horas de trabalho), o projeto estaria cobrindo todos os 9 critérios de avaliação. O trabalho realizado desde o último review foi expressivo: 4 dos 5 critérios que estavam completamente ausentes foram implementados. O foco agora são os ajustes de segurança, responsividade dos dashboards, deploy e documentação.